Name :- Sheikh Mohd Sabir

Program Name :- B.Tech (Computer Engineering)

Semester :- 6th semester

Examination Roll No. :- 19BCS058

Unique Paper Code :- CEN-602

Paper Title :- Software Engineering

Date & Time of Examination :- 28/05/2022 (10:00 am to 1:00 pm)

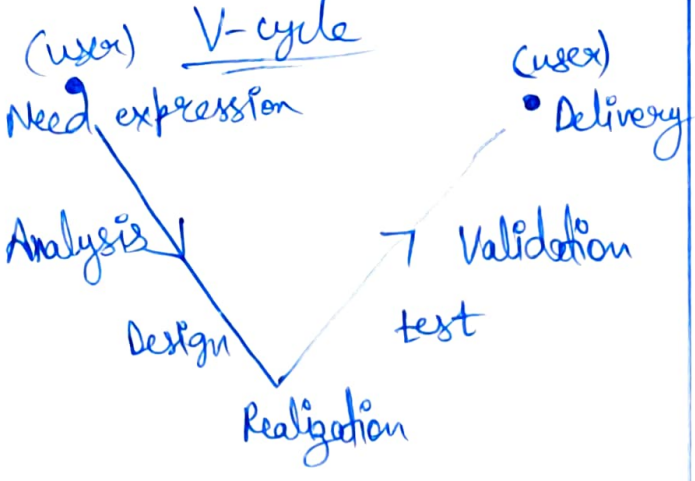
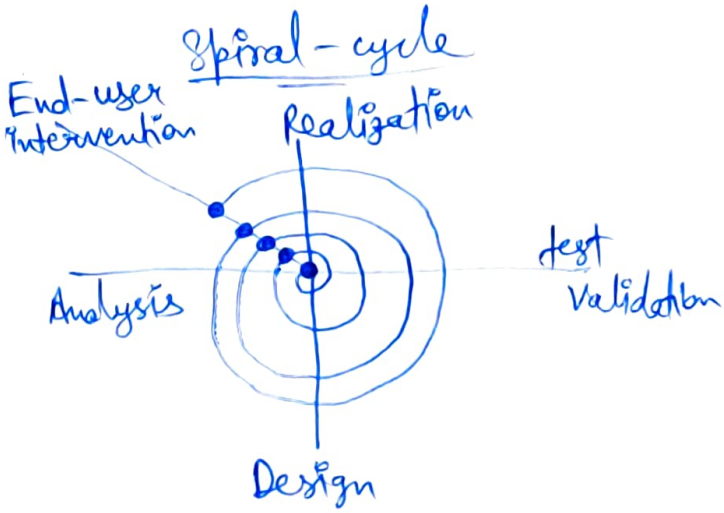
Sol 1(a) Prototyping basically means to practically implement the findings of a discovery phase in design development. It means to develop a model, etc a dummy website for a product that is partially functional & gives a great tangible idea of how the product will look and feel after its development.

Effect of designing a prototype on the overall cost of the software project :-

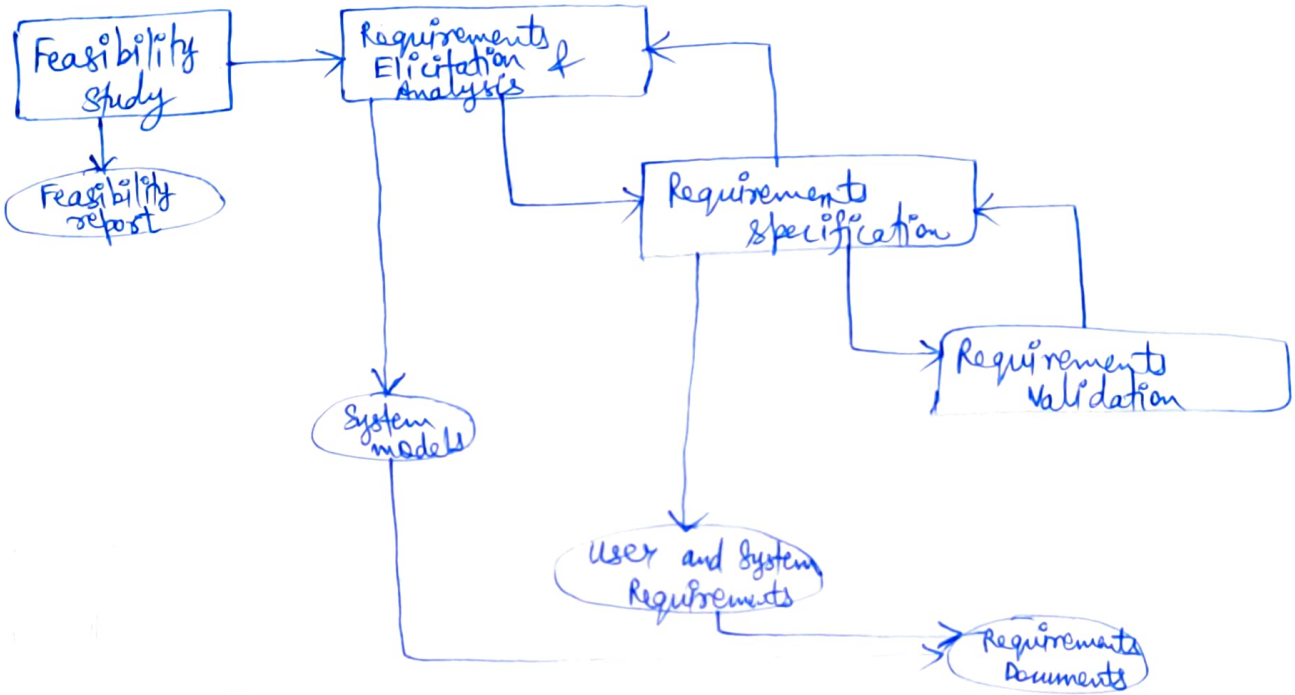
Prototype may have some initial costs of developing but it reduces the overall budget by helping our product to be free of the errors or glitches that could have been occurred if the idea was made from scratch without any prior user testing. Furthermore, prototyping also helps to understand the intrinsic flaws, shortcomings & drawbacks that can be improved during the product development process. If the prototyping process is ignored completely, it might result in the restructuring and redesigning of the entire product after spending all your resource on its development.

So, the effect of designing a prototype on the overall cost of a software project is to actually reduce the additional costs of restructuring & reframing it after its full-fledged development which might cost a fortune.

Sheikh Mohd Sabir, 19BCS058

V model	SPIRAL MODEL
(1.) 	
(2.) Vmodel is software development model but development & testing are not concurrent.	Spiral model is a software development model and is made with features of incremental, waterfall, or evolutionary prototyping models.
(3.) Testing activities start with the first stage.	Testing is done at the end of the engineering phase
(4.) Flexibility of V-model is Little flexible	Flexibility to change in spiral model is not that difficult.
(5.) Guarantee of success through V-model is high.	Guarantee of success through spiral model is Low.
(6.) It is not iterative.	It is iterative

Sol 1(b.) The process of defining, documentation and maintenance of requirements in the design process of engineering is called requirements engineering. It provides an apt mechanism to understand the customer's desires, analysis of needs of customer, feasibility etc. In contrast, the requirements are being transferred to the working system. Hence, this makes the requirements engineering an application that is disciplined, having the principles, methods, etc. that are proved.



Steps involved in Requirements Engineering :-

(1.) Feasibility Study :-

The main aim of a feasibility study is to create reasons for the development of the software that users accept, that is feasible enough and open to changes, and abide by the standards chosen for software development and maintenance.

Several types of feasibility :-
1. Technical Feasibility
2. operational Feasibility
3. Economic Feasibility

(2.) Elicitation of Requirements & Analysis :-

This process is also called requirements gathering. If there are any existing processes available and with the help of customers, requirements gathering is done Elicitation of requirements is the starting step of requirements analysis.

(3.) Specification of Software Requirements :-

A document consisting of requirements that are collected from various sources like the requirements from customers expressed in an ordinary language and created by the software analyst is called a specification document for software requirements. Several models are used during the process of specification of software requirements like Entity - Relationship diagrams (ER diagrams), data flows diagram (DFD), etc.

(4.) Validation of Software Requirements :-

After the development of the specification of requirements, the requirements laid down in the document are validated. There may be requirements from the users that are illegal or which cannot be accomplished, or the experts can misunderstand the needs.

(5.) Management of Software Requirements :-

The process of managing the requirements that keep changing during the process of requirements engineering & development of the system is called management of software requirement. During the process of software management, there are new requirements with the change in the needs of business, etc. During the process of development, the technical & business environment of the system changes.

It is essential to follow these steps as it comes with many advantages as :-

- Lesser chances of process overhead.
- The requirements that are critical can be identified and implemented in the initial stage using requirements engineering
- The identification & documentation of requirements undergo a series of iteration before finalisation. Hence, the response to the changes is quick and cost becomes low.

FAST :-(Fasilitated Application Specification Technique)

A team of oriented approach is developed for need gathering & is called facilitated application specification techniques (FAST).

This approach supports the creation of a joint team of customers and developers who work together to understand better the expectations and purpose of set of requirements.

The fundamental guidelines of FAST are :-

→ arrange a meeting at a neutral site for dev's & customers.
→ Establishment of rules for preparation & participation.
→ Prepare as well an informal agenda that encourages free flow of ideas.
→ Participants must not criticise or debate.
→ A facilitator should be present to manage meeting.

Sheikh Mohd Sabir, 19BCS0058

- Now, we know that both Brainstorming sessions and FAST are methods of requirement elicitation.
- Both are a group of techniques.
- Brainstorming is intended to generate lots of new ideas, whik each idea is documented & discussed.
- A trained facilitator is used to handle the group conflicts & group bias, and A document is prepared which consists of the list of requirements and their priority if possible.
- While fast, the objective is to bridge the gap b/w developers & customers.
- A Draft of specifications is written down using all the inputs from meeting.

Sheikh Mohd Sabir, 19BCS058

Sol 2(a)

	Intermediate COCOMO	Detailed COCOMO
1.	The basic COCOMO assumes that the effort is only a function of the number of Lines of code and some constants evaluated according to the different software systems. However, in reality, no system is effort can be calculated only on the basis of LOC. Hence, Intermediate COCOMO takes in consideration of various cost drivers (factors) such as capability, experience etc.	It encorporates all the characteristics of the intermediate version with an assessment of cost drivers impact on each step of software engineering process. Detailed model uses diff. effort multipliers for each cost drive attributes
2.	Classification of cost drivers :- (a) Product Attributes (b) Hardware attributes (c) Personnel attributes (d) Project attributes	Six phases of this model :- (a) Planning & requirements (b) System Design (c) Detailed design (d) module code & test (e) Integration & test (f) Cost construction model.
3.	$E = (a(KLOC)^b) * EAF$	Effort is calculated as functions of program set and set of cost drivers.

Sheikh Mohd Sabir, 19BCS058

KLOC = 200 KLOC

$E = 3.0 (200)(1.12) = 1133.12$ PM

$D = 2.5 (1133.12)(0.35) = 29.3$ PM

overall staff size (SS) $= \frac{E}{D} = \frac{1133.12}{29.3} = 38.67$ persons

Productivity (P) $= \frac{KLOC}{E} = \frac{200}{1133.12} = 0.1765$ KLOC/PM

$\Rightarrow P = 176$ LOC/PM

Sol 2 (b:) A risk is a potential problem. It's an activity or event that may compromise the success of software development project. Risk is the possibility of suffering loss and total risk exposure to a specific project will account for both the probability & the size of the potential loss. Risk management means risk containment & mitigation.

Risk management includes following tasks:-

- Identify – Identification of risks & their triggers.
- Classify and prioritize all risks.
- Craft a plan that links each risk to a mitigation.
- Monitor for risk triggers during the project.
- Implement the mitigation action if any risk materializes.
- Communicates risk status throughout project.

Sheikh Mohd Sabir, 19BCS058

	Function oriented Design	Object oriented Design
1.	Function oriented design is the result of focussing attention to the function of the program. This is based on stepwise refinement.	Object oriented design is the result of attention not the function performed by the program, but instead on the data that are to be manipulated by the program.
2.	The basic abstractions which are given to users are real world functions	The basic abstractions are not the real world Functions, but are the data abstraction where the real world entities are presented.
3.	Carried out using structured analysis & structured design, i.e, data flow diagram	Carried out using UML.
4.	It is top-down approach.	It is bottom-up approach.
5.	Begins by considering use case diagrams & the scenarios.	Begins by identifying objects & classes.
6.	In function oriented design, we decompose in function/ procedural level.	We decompose in class level, here.

Sol 3(a) Module Coupling :-

This refers to couple two to more modules with each other and with the outside world. It generally represents how the modules are connected with another module and the outside world. Coupling is related to cohesion.
Cohesion means that the cohesive module performs only one task or one thing in the overall software procedure with a small amount of interaction with other modules.

Types of Cohesion :-

* Functional Cohesion :-
A functional cohesion performs the tasks & functions. It is an ideal situation.

* Sequential Cohesion :-
An element outputs some data, that becomes the input for other element. ie, data flow b/w the parts.

* Communicational Cohesion :-
Two elements operate on the same input data or contribute towards the same output data.
eg :- update record in the dB and send it to printer.

* Procedural Cohesion :-
Elements of procedural cohesion ensures the order of execution.
eg :- calculate student GPA, print student record, etc

* Temporal Cohesion :-
This cohesion contains the code for initializing all the parts of the system.

* Logical Cohesion :-
The elements are logically related & not functionally
Eg:- A component reads inputs from tape, disk and network. Operations are related but the functions are significantly different.

Advantages of Modular Programming :-

- Development can be divided
- Readable Programs
- Programming Errors are easy to Detect
- Improves manageability
- Collaboration
- Allows Re-use of codes.

Sol 3(b.) Software Reliability

↳ It means operational reliability. It is described as the ability of a system or component to perform its required functions under ~~static~~ condition for a specific period.

It is also defined as the probability that a software system fulfills its assigned task in a given environment for a predefined number of input cases, assuming that the hardware & the input are free of errors.

- Software Reliability is hard to achieve due to complexity of software whereas the hardware Reliability is better than the software.
- Hardware failure is due to the physical changes that takes place in hardware over a period of time i.e., the components deteriorates (due to overheating, etc). However, there is not such wear out phenomenons in case of software.
- As compared to hardware software can be replicated to high standards of quality.



- The concept of failure is fundamental to reliability whereas, the adjudged cause of a failure is called fault.
- Failures are either random or systematic in nature whereas, the faults are initiated through a developer making an error / human errors, etc.

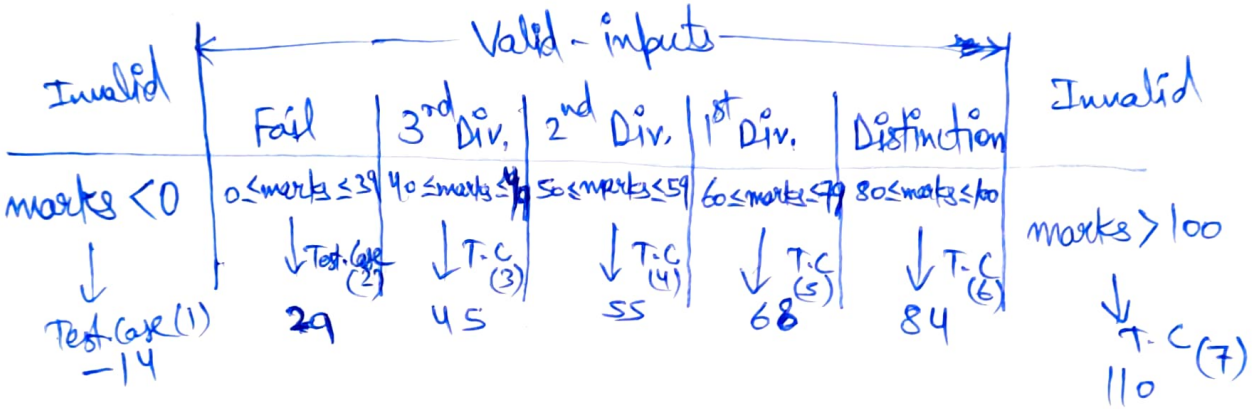
Sheikh Mohd Sabir, 19BCS058

Sol 4(a.) Equivalence Class Testing

The grading is done acc. to question :-

Grade	Marks Obtained
Distinction	80 - 100
First Division	60 - 79
Second Division	50 - 59
Third Division	40 - 49
Fail	30 - 39

Accepted Inputs :- b/w 0 to 100
Not accepted inputs :- less than 0 or more than 100



Test Scenario	Description	Expected output	Output
1	marks less than zero	Invalid	—
2	marks b/w 0 to 39	valid	Fail
3	marks b/w 40 to 49	valid	3rd div.
4	marks b/w 50 to 59	valid	2nd div.
5	marks b/w 60 to 79	valid	1st div.
6	marks b/w 80 to 100	valid	distinction
7	marks greater than 100	Invalid	—

Sheikh Mohd Sabir, 19BCS058

Sol 4(b)

	Code Inspection	Code Walkthrough
(1.)	It is formal.	It is informal
(2)	Initiated by project team.	Initiated by author.
(3.)	A group of relevant persons from different departments participate in the inspection.	Usually team members of the same project take participation in the walkthrough. Author himself acts walkthrough Leader.
(4.)	Checklist is used to find faults	No checklist used in the walkthrough
(5)	Recorder records the defect.	Author make a note of defects & suggestions offered by teammate

Sheikh Mohd Sabir, 19BCS058

Black-box testing	White-box testing	Grey-box testing
• Knowledge of internal working structure (code) is not required for this type of testing Only GUI is required for test cases.	Knowledge of internal working structure is necessarily required for this type of testing	Partial knowledge of the internal working structure is required.
• It is also known as functional testing, data-driven testing closed testing	It is also known as Structural testing, Clear-box testing, code-based testing & transparent testing	It is also known as translucent testing as the tester has limited knowledge of coding.
• It is not considered for algorithm testing	It is well suitable for algorithm testing	It is not considered for algorithm testing

Sol 5. • Types of Maintenance

Software maintenance is the process of modifying a software product after it has been delivered to the customer. Software maintenance performed to correct faults, improve design, implement enhancements & updates, etc.
The maintenance is basically divided into 4 categories:-

(1.) Correct Maintenance :-

Corrective maintenance of a software product may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.

(2.) Adaptive Maintenance :-

This includes modifications and updations when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interfere/interface with new hardware and software.

(3.) Perfective Maintenance :-

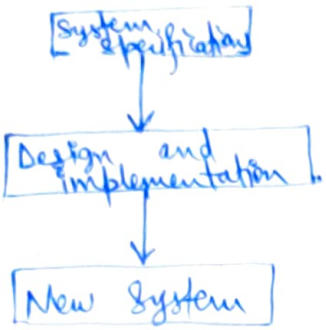
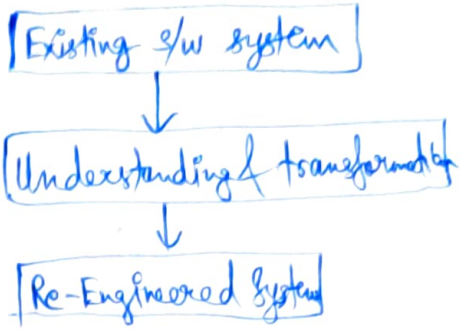
A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands in the market.

(4.) Preventive maintenance :-

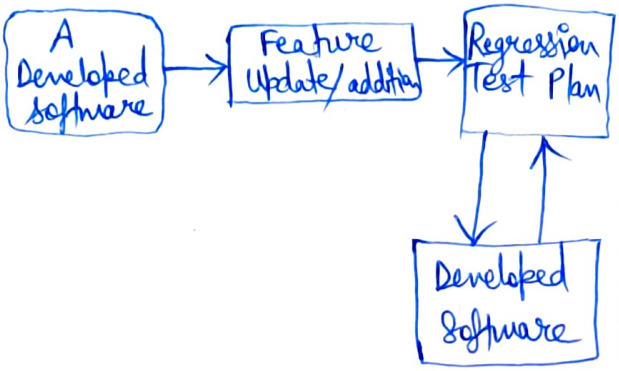
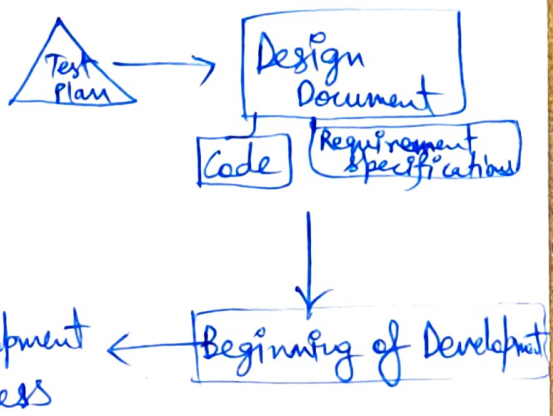
This type of maintenance includes modifications & updations to prevent future problems of the software. It goals to attend problems, which are not significant at this moment but may cause serious issues in future.

Sheikh Mohd Sabir, 19BCS058

Sol 5 • New Software Development Vs Re-Engineering

	New Software Development	Re-Engineering
(1.)	New Software Development involves building of whole software from scratch.	It involves transformation & improvement of previously created software to an upgraded one.
(2.)	 It is also known as the Forward Engineering.	
(3.)	It is the starting point of development.	It uses old system as the specification for the new system.
(4.)	Higher Risk, as many bugs & problems comes in this phase.	Reduced Risk, as the software is already existing and just need an improvement.
(5.)	Higher cost, as everything starts with ground 0 here.	Reduced cost in compare to new software development.
(6.)	More maintainable, as the tools and technologies used are modern.	Less maintainable, as it is done using the tech already used in the old system.

sol 5. • Regression testing vs Development Testing :-

	Regression Testing	Development Testing
1.	This type of testing is done when any new change occurs in part of the existing code.	It is the concept of applying testing practices consistently throughout the software development life cycle model.
2.		
3.	It is the process of analysing any deviations.	It is a continous process that runs throughout the development.
4.	It is done to check & ensure that the same software product works as efficiently as it was earlier.	It is done to ensure timely & efficient delivery of a software system.
5.	It is performed several times over the life of a software product.	Development testing is performed only once.
6.	Regression testing is often performed in crisis situation.	Development Testing is often carried out to meet the deadline date of software delivery.